
Java RMI

Mục đích

- ❑ Hỗ trợ gọi phương thức từ xa trên các đối tượng trong các máy ảo khác nhau
- ❑ Tích hợp mô hình đối tượng phân tán vào Java
- ❑ Làm cho sự khác biệt giữa mô hình đối tượng phân tán và mô hình đối tượng cục bộ không có sự khác biệt.
- ❑ Tạo ra các ứng dụng phân tán có độ tin cậy một cách dễ dàng
- ❑ Duy trì sự an toàn kiểu được cung cấp bởi môi trường thời gian chạy của nền tảng Java

Mô hình gọi đối tượng từ xa - RMI

Local Machine (Client)

```
SampleServer remoteObject;  
int s;  
...  
s = remoteObject.sum(1,2);
```

```
System.out.println(s);
```

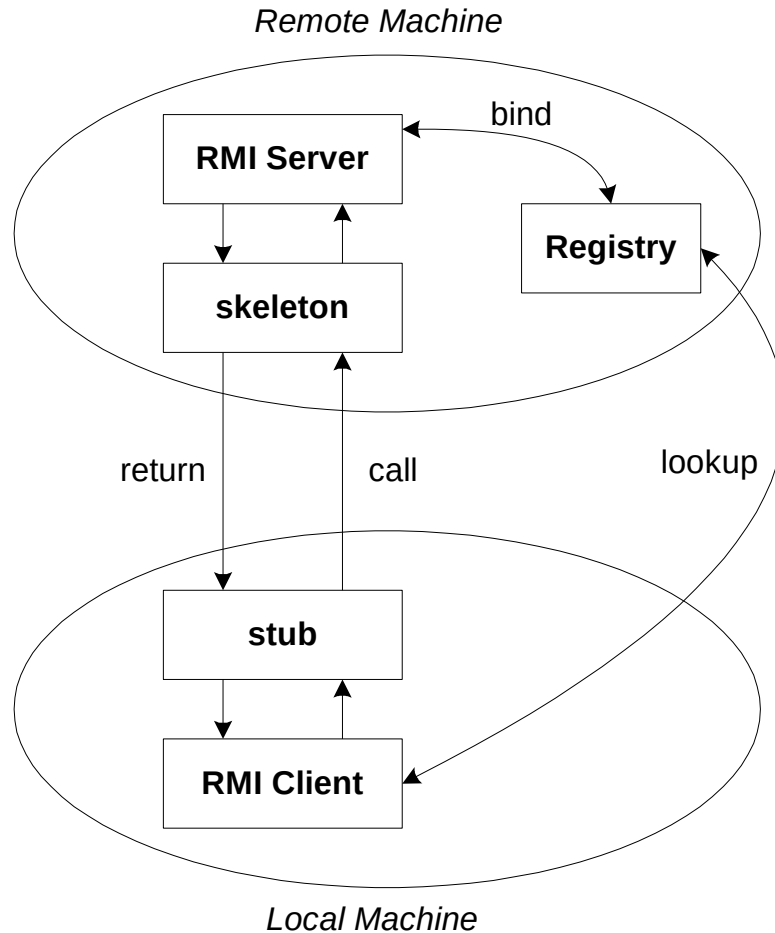
1,2

3

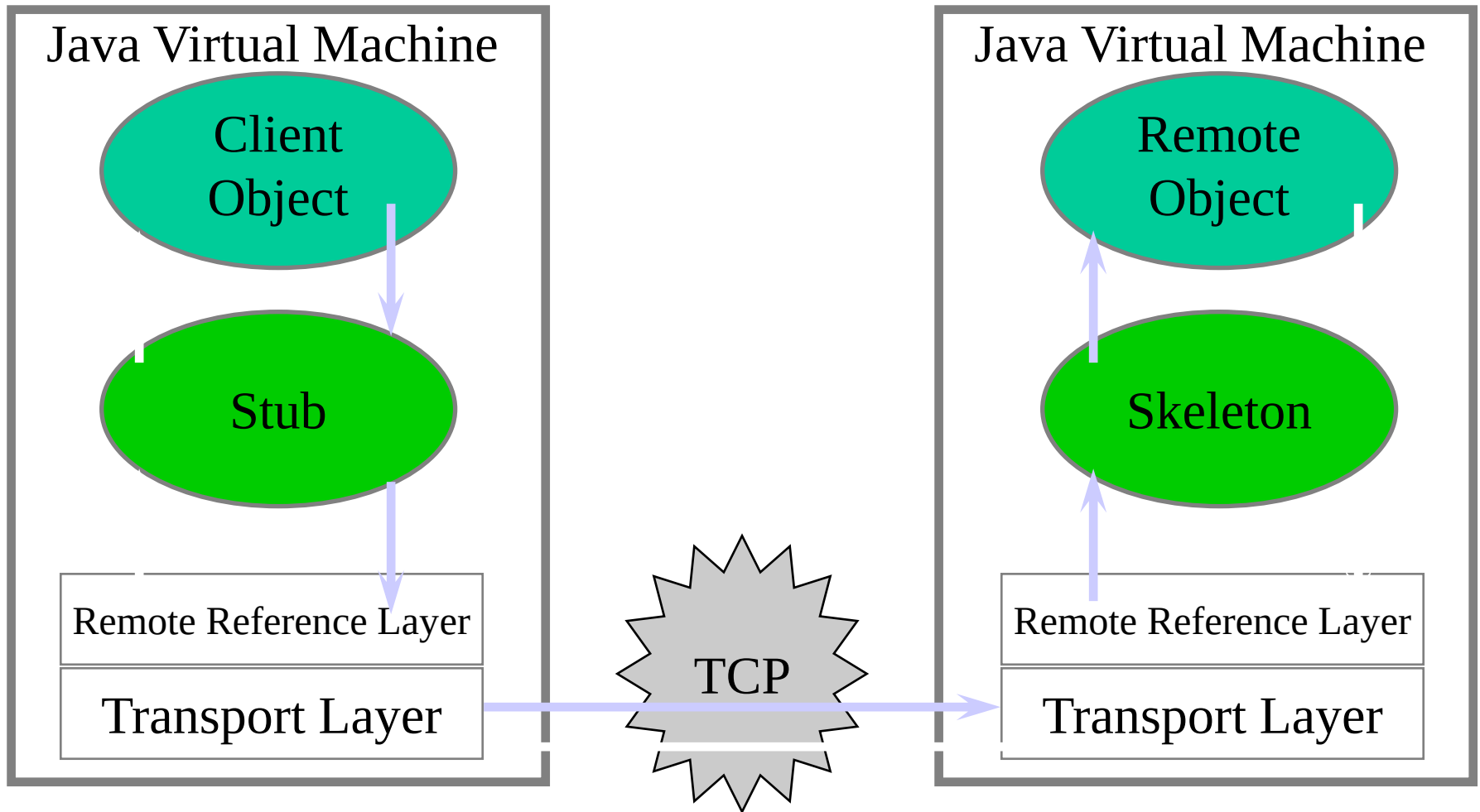
Remote Machine (Server)

```
public int sum(int a,int b)  
{  
    return a + b;  
}
```

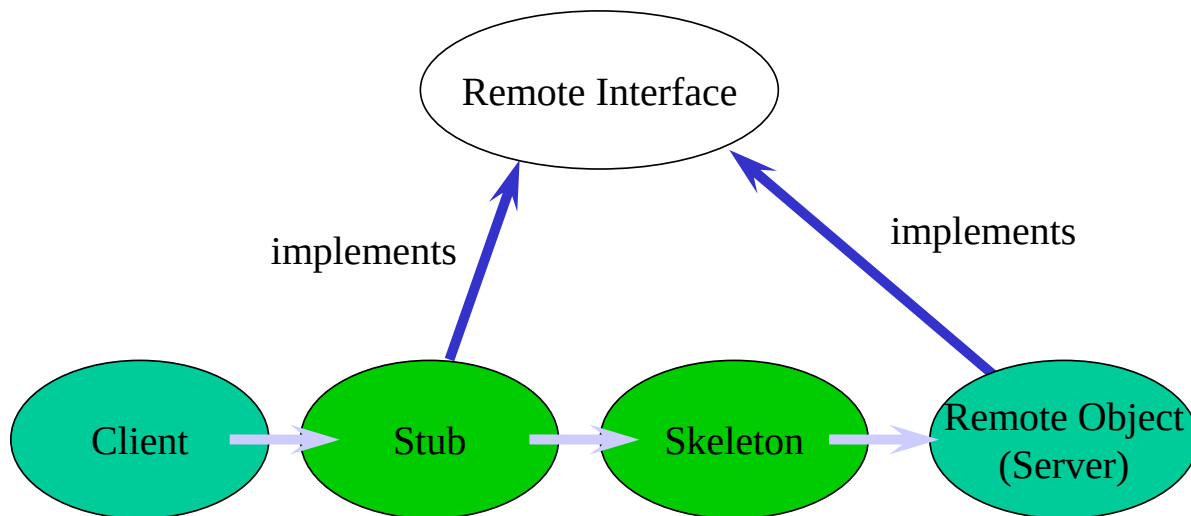
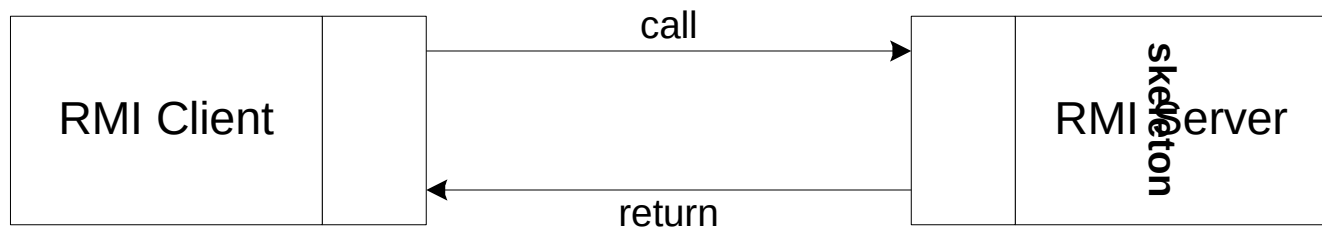
Kiến trúc RMI



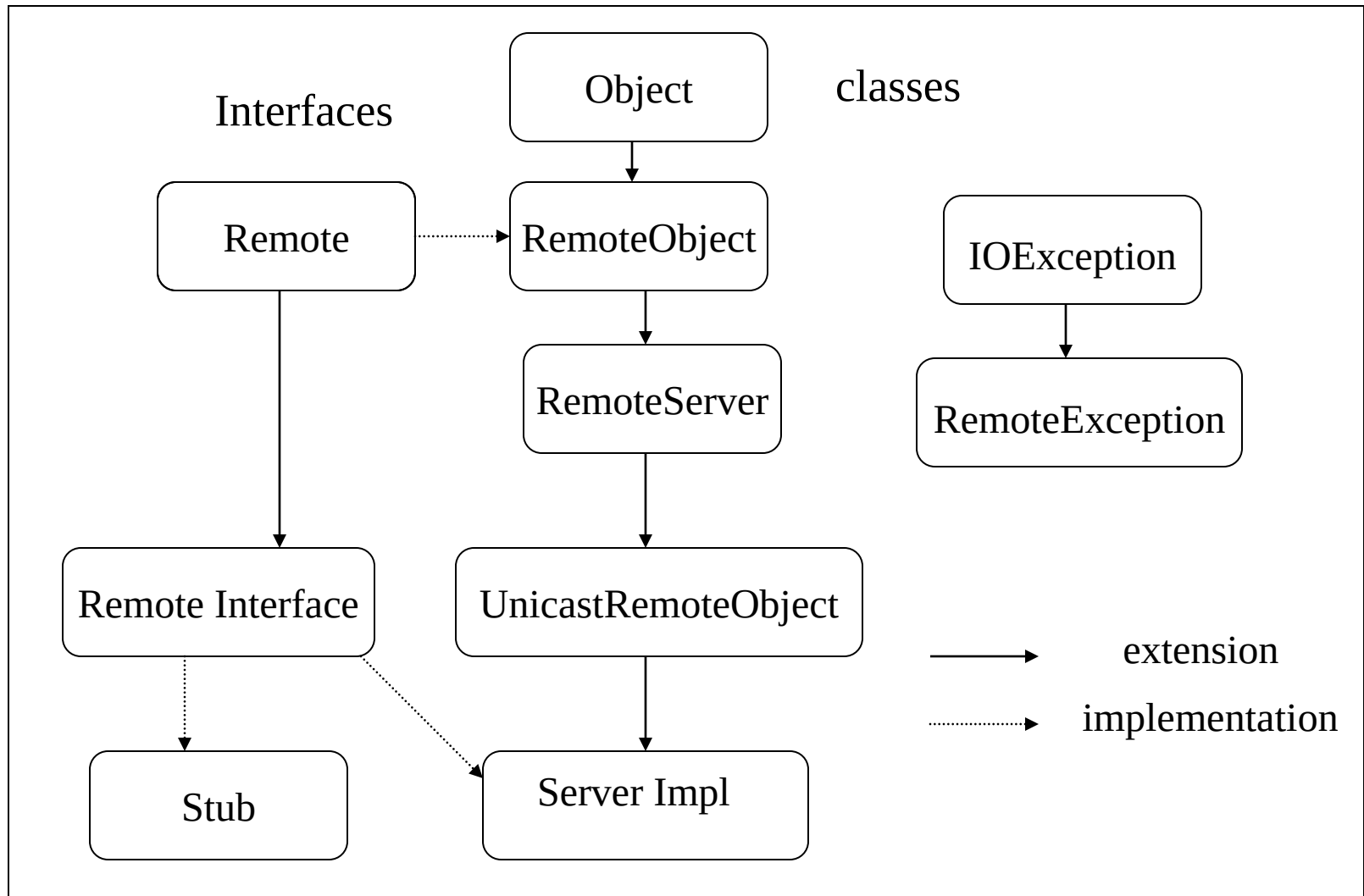
Java RMI Layers



The Stub and Skeleton



Mô hình các đối tượng phân tán



Các bước tạo RMI

1. Định nghĩa giao diện
2. Phát triển đối tượng bằng cách cài đặt giao diện remote
3. Phát triển chương trình client, server.
4. Biên dịch tập tin java.
5. Tạo ra đối tượng stub và skeleton.
6. Khởi động RMI registry.
7. Chạy đối tượng remote server
8. Chạy client

Ví dụ 1

Chương trình tính tổng của 2 số nguyên nhập từ bàn phím sử dụng mô hình RMI.

Bước 1. Định nghĩa giao diện

```
8  import java.rmi.*;  
9  
10 public interface ICalculator extends Remote {  
11  
12     public int addNum(int x, int y) throws RemoteException;  
13 }
```

Bước 2. Phát triển đối tượng bằng cách cài đặt giao diện remote

```
8 import java.rmi.*;
9 import java.rmi.server.UnicastRemoteObject;
10
11 public class CalculatorImpl extends UnicastRemoteObject implements ICalculator {
12
13     public CalculatorImpl() throws RemoteException {
14         super();
15     }
16
17     @Override
18     public int addNum(int x, int y) throws RemoteException {
19         System.out.println("Client request to Calculator!!");
20         return (x + y);
21     }
22 }
```

Bước 3. Phát triển chương trình server.

```
8  import java.rmi.*;
9  import java.rmi.registry.LocateRegistry;
10 import java.rmi.registry.Registry;
11
12 public class CalculatorServer {
13
14     public static void main(String[] args) {
15         try {
16             CalculatorImpl c = new CalculatorImpl();
17             Registry r = LocateRegistry.createRegistry(34560);
18             r.bind("MyCalculator", c);
19         } catch (AlreadyBoundException | RemoteException e) {
20             System.out.println(e);
21         }
22     }
23 }
```

Bước 4. Phát triển chương trình client.

```
8 import java.util.*;
9 import java.rmi.*;
10 import java.rmi.registry.LocateRegistry;
11 import java.rmi.registry.Registry;
12 import java.util.logging.Level;
13 import java.util.logging.Logger;
14
15 public class CalculatorClient {
16
17     public static void main(String[] args) throws NotBoundException {
18         try {
19             Registry r = LocateRegistry.getRegistry("127.0.0.1", 34560);
20             ICalculator c = (ICalculator) r.lookup("MyCalculator");
21             Scanner in = new Scanner(System.in);
22             System.out.print("a= ");
23             int a = in.nextInt();
24             System.out.print("b= ");
25             int b = in.nextInt();
26             System.out.println("Result: " + c.addNum(a, b));
27         } catch (RemoteException ex) {
28             Logger.getLogger(CalculatorClient.class.getName()).log(Level.SEVERE, null, ex);
29         }
30     }
31 }
```

Bước 5. Thực thi

1. Thực thi chương trình server
2. Thực thi chương trình client

Ví dụ 2

Chương trình server quản lý thông tin tài khoản người dùng ATM:

- Thông tin người dùng gồm có: tên, CMND/CCCD, mật khẩu, số tiền, lãi suất, thông tin các giao dịch trên tài khoản.
- Thông tin giao dịch gồm: giao dịch, số tiền, kiểu giao dịch (rút tiền hoặc gửi tiền).

Chương trình có các đối tượng quản lý cho phép triệu gọi từ xa bởi các máy ATM (mô hình RMI):

- Thêm một người sử dụng mới vào danh sách để quản lý.
- Tìm người sử dụng có tên và mật khẩu
- Cho phép nhập/rút tiền trên tài khoản có tên và mật khẩu.